

LangChain

1. What is LangChain?

LangChain = framework for building LLM applications

- **Framework** = reusable building blocks
 - **LLM** = language model integration
 - **Tools** = external capabilities
 - **Retrievers** = fetch relevant information
 - **Purpose** = connect LLMs with data, tools, and workflows
-

2. Why do we need LangChain?

LangChain = simplifies LLM application orchestration

- **Without LangChain** = manually integrate models, prompts, tools, retrieval
 - **With LangChain** = use reusable components
 - **Benefit** = faster development
 - **Trade-off** = additional abstraction (layer hiding implementation details)
-

3. What are the main components of LangChain?

LangChain = models + prompts + retrieval + tools + agents

- **Models** = interact with LLMs
 - **Prompts** = structure model input
 - **Retrievers** = fetch relevant data
 - **Tools** = perform external actions
 - **Agents** = decide which actions to take
-

4. What is a Chat Model in LangChain?

Chat Model = interface for conversational LLMs

- **Input** = messages
 - **Messages** = system, human, AI
 - **Output** = AI message
 - **Example** = GPT, Claude, Gemini
-

5. What is a prompt template?

Prompt Template = reusable structure for prompts

- **Template** = fixed instructions + variables
 - **Variable** = dynamic input
 - **Example** = `{question}`
 - **Benefit** = consistent prompts
-

6. What is a chain in LangChain?

Chain = sequence of connected operations

- **Input** = user request
 - **Step 1** = create prompt
 - **Step 2** = call LLM
 - **Step 3** = process output
 - **Purpose** = create predictable workflows
-

7. What is LCEL?

LCEL = LangChain Expression Language

- **Purpose** = compose LangChain components
- **Syntax** = uses runnable pipelines
- **Example** = `prompt | model | parser`
- **Benefit** = composable and streamable workflows

8. What is a Runnable?

Runnable = standard interface for executable LangChain components

- **Input** = data
 - **Process** = component logic
 - **Output** = transformed data
 - **Examples** = prompt, model, parser, retriever
-

9. What is an output parser?

Output Parser = converts LLM output into usable data

- **Input** = model response
 - **Output** = structured application data
 - **Example** = JSON, list, string
 - **Benefit** = easier downstream processing
-

10. What is a tool in LangChain?

Tool = function an LLM can invoke

- **Function** = performs actual operation
 - **Input schema** = defines required arguments
 - **Example** = database search
 - **Example** = weather API
 - **Result** = returned to LLM
-

11. What is an agent in LangChain?

Agent = LLM that decides which tools to use

- **LLM** = reasoning engine
- **Tools** = available actions

- **Decision** = selects appropriate tool
 - **Loop** = think → act → observe
 - **Goal** = complete the requested task
-

12. Chain vs Agent?

Chain = predefined workflow, Agent = dynamic workflow

- **Chain** = fixed sequence
 - **Agent** = decides next action
 - **Chain** = predictable
 - **Agent** = flexible
 - **Use Chain** = known workflow
 - **Use Agent** = tool selection requires decisions
-

13. What is a Retriever?

Retriever = component that fetches relevant documents

- **Input** = user query
 - **Search** = finds relevant information
 - **Output** = documents/chunks
 - **Used in** = RAG
-

14. What is a Vector Store?

Vector Store = stores and searches embeddings

- **Embedding** = numerical representation of text
 - **Storage** = vectors + metadata
 - **Search** = similarity search
 - **Examples** = Pinecone, Qdrant, Chroma, FAISS
-

15. Retriever vs Vector Store?

Vector Store = storage/search, Retriever = retrieval interface

- **Vector Store** = stores embeddings
 - **Retriever** = asks for relevant documents
 - **Retriever** = can use vector store internally
 - **Benefit** = separates retrieval logic from storage
-

16. How does RAG work in LangChain?

LangChain RAG = load → split → embed → store → retrieve → generate

- **Loader** = reads source documents
 - **Splitter** = creates chunks
 - **Embedding** = converts chunks to vectors
 - **Vector Store** = stores vectors
 - **Retriever** = finds relevant chunks
 - **LLM** = generates final answer
-

17. What is a Document in LangChain?

Document = content plus metadata

- **Content** = actual text
 - **Metadata** = source, page, ID, etc.
 - **Used by** = retrievers and document loaders
 - **Purpose** = preserve content context
-

18. What is a Document Loader?

Document Loader = loads data into LangChain documents

- **Sources** = PDF, web page, CSV, database
 - **Output** = Document objects
-

- **Purpose** = standardize different data sources
-

19. What is a Text Splitter?

Text Splitter = divides large documents into smaller chunks

- **Chunk** = smaller retrievable text section
 - **Chunk size** = maximum chunk length
 - **Overlap** = shared text between chunks
 - **Purpose** = improve retrieval and context management
-

20. What is chunk overlap?

Chunk overlap = shared content between adjacent chunks

- **Purpose** = preserve context
 - **Example** = chunk 1 ends with sentence X
 - **Overlap** = chunk 2 also contains sentence X
 - **Trade-off** = better context but more storage/tokens
-

21. What is Conversational RAG?

Conversational RAG = RAG that understands previous conversation

- **Chat history** = previous messages
 - **Query rewriting** = converts follow-up into standalone query
 - **Retriever** = searches using rewritten query
 - **LLM** = generates contextual answer
-

22. What is memory in LangChain?

Memory = mechanism for maintaining conversation context

- **Short-term memory** = current conversation context
- **Long-term memory** = information persisted beyond conversation

- **Purpose** = maintain context
 - **Production concern** = token cost and privacy
-

23. What is LangGraph?

LangGraph = framework for stateful agent workflows

- **Graph** = nodes + edges
 - **Node** = performs an operation
 - **Edge** = controls next step
 - **State** = shared workflow information
 - **Use** = complex, multi-step agents
-

24. LangChain vs LangGraph?

LangChain = components, LangGraph = workflow orchestration

- **LangChain** = models, prompts, tools, retrievers
 - **LangGraph** = stateful execution flow
 - **Simple workflow** = LangChain
 - **Complex agent workflow** = LangGraph
-

25. What is streaming in LangChain?

Streaming = return model output incrementally

- **Without streaming** = wait for complete response
 - **With streaming** = receive tokens/chunks progressively
 - **Benefit** = lower perceived latency
 - **Use** = chat applications
-

26. What is batching?

Batching = process multiple inputs together

- **Input** = multiple requests
 - **Execution** = grouped processing
 - **Benefit** = better resource utilization
 - **Use** = embeddings, bulk LLM processing
-

27. How does LangChain handle multiple LLM providers?

LangChain = common interface across model providers

- **Provider** = OpenAI, Anthropic, Google, etc.
 - **Interface** = common LangChain abstraction
 - **Benefit** = easier provider switching
 - **Trade-off** = provider-specific features may need direct APIs
-

28. What are callbacks in LangChain?

Callbacks = hooks into execution events

- **Events** = model start, model end, tool call
 - **Purpose** = logging and monitoring
 - **Use** = token tracking
 - **Use** = debugging and tracing
-

29. What is LangSmith?

LangSmith = observability and evaluation platform for LLM applications

- **Tracing** = tracks execution steps
 - **Debugging** = inspect prompts and outputs
 - **Evaluation** = measure application quality
 - **Monitoring** = observe production behavior
-

30. How would you build a production RAG application using LangChain?

Production RAG = ingestion + retrieval + generation + observability

```
Documents
  ↓
Document Loader
  ↓
Text Splitter
  ↓
Embeddings
  ↓
Vector Store
  ↓
Retriever
  ↓
Prompt
  ↓
LLM
  ↓
Response
```

- **Ingestion** = process documents
- **Retrieval** = find relevant context
- **Generation** = LLM creates answer
- **Caching** = reduce repeated computation
- **Observability** = trace latency, tokens, and failures

31. When would you NOT use LangChain?

Don't use LangChain when the abstraction doesn't add value

- **Simple LLM call** = direct provider SDK may be enough
- **Simple RAG** = custom implementation may be simpler
- **Performance-critical path** = direct control may help
- **Complex workflow** = LangGraph may be more suitable
- **Principle** = use abstraction when it reduces complexity, not automatically